

**ASSIGNMENT No: 11****TITLE: Write** a Java program to demonstrate the implementation of interfaces.

Problem Statement

Write a Java program to demonstrate the implementation of interfaces.

Learning Objectives

To implement interfaces for achieving abstraction and multiple inheritance of type.

Theory

An interface specifies a set of abstract methods that implementing classes must define. It provides complete abstraction and supports multiple inheritance of type. Interfaces improve modularity, loose coupling, and software flexibility. They allow different classes to implement common behavior while maintaining independent implementations. Interfaces are widely used in enterprise application development.

Code

```
interface Vehicle {  
  
    void start();  
    void stop();  
}  
  
class Car implements Vehicle {  
  
    public void start() {  
        System.out.println("Car starting");  
    }  
  
    public void stop() {  
        System.out.println("Car stopped.");  
    }  
}  
  
class Bike implements Vehicle {  
  
    public void start() {  
        System.out.println("Bike starting");  
    }  
  
    public void stop() {  
        System.out.println("Bike stopped.");  
    }  
}
```

```

}

public class Demo{
    public static void main(String[] args) {

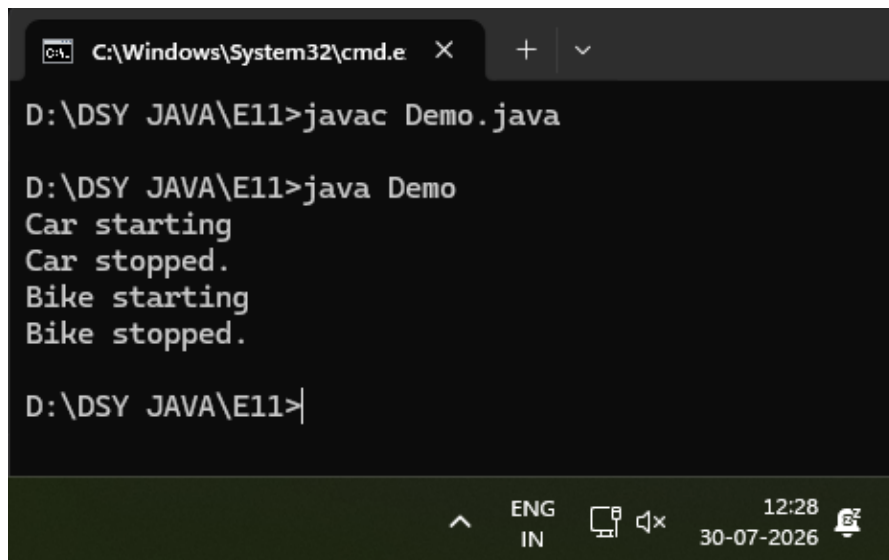
        // Vehicle v1=new Vehicle(); error: Vehicle is abstract; cannot be instantiated

        Vehicle v1 = new Car();
        v1.start();
        v1.stop();

        Vehicle v2 = new Bike();
        v2.start();
        v2.stop();
    }
}

```

Output



```

C:\Windows\System32\cmd.e
D:\DSY JAVA\E11>javac Demo.java

D:\DSY JAVA\E11>java Demo
Car starting
Car stopped.
Bike starting
Bike stopped.

D:\DSY JAVA\E11>

```

Exercise

1. Create an interface **Printable** and implement it in Student and Employee classes.

Code

```

interface Printable {
    void print();
}

class Student implements Printable {
    private String name;
    private int rollNumber;

    public Student(String name, int rollNumber) {
        this.name = name;
    }
}

```

```

        this.rollNumber = rollNumber;
    }

    public void print() {
        System.out.println("Student Details");
        System.out.println("Name: " + name);
        System.out.println("Roll Number: " + rollNumber);
        System.out.println();
    }
}

class Employee implements Printable {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public void print() {
        System.out.println("Employee Details");
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
        System.out.println();
    }
}

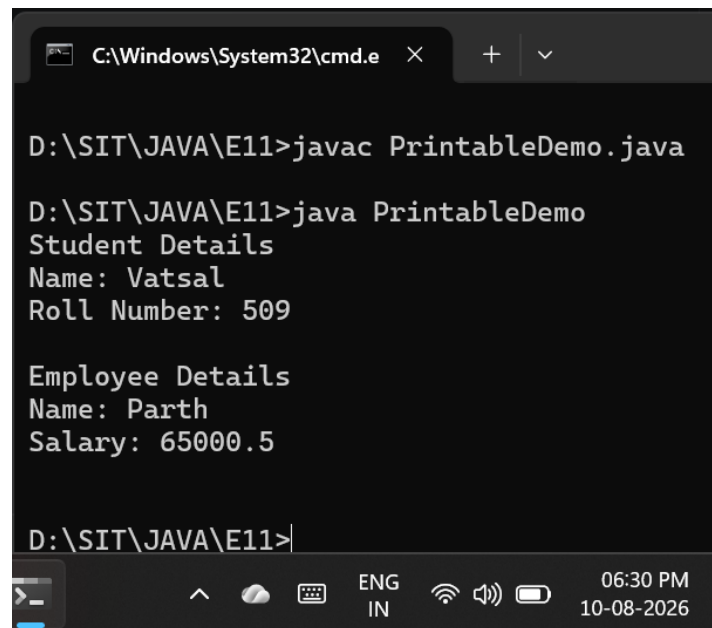
public class PrintableDemo {
    public static void main(String[] args) {

        Printable student = new Student("Vatsal", 509);
        Printable employee = new Employee("Parth", 65000.50);

        student.print();
        employee.print();
    }
}

```

Output



```
C:\Windows\System32\cmd.e X + v

D:\SIT\JAVA\E11>javac PrintableDemo.java

D:\SIT\JAVA\E11>java PrintableDemo
Student Details
Name: Vatsal
Roll Number: 509

Employee Details
Name: Parth
Salary: 65000.5

D:\SIT\JAVA\E11>
```

2. Create an interface **Switchable** with a method `turnOn()`. Implement the interface in **Light** and **Fan** classes to display the device status.

Code

```
interface Switchable {
    void turnOn();
}

class Light implements Switchable {
    public void turnOn() {
        System.out.println("Status: The Light is now turned ON.");
    }
}

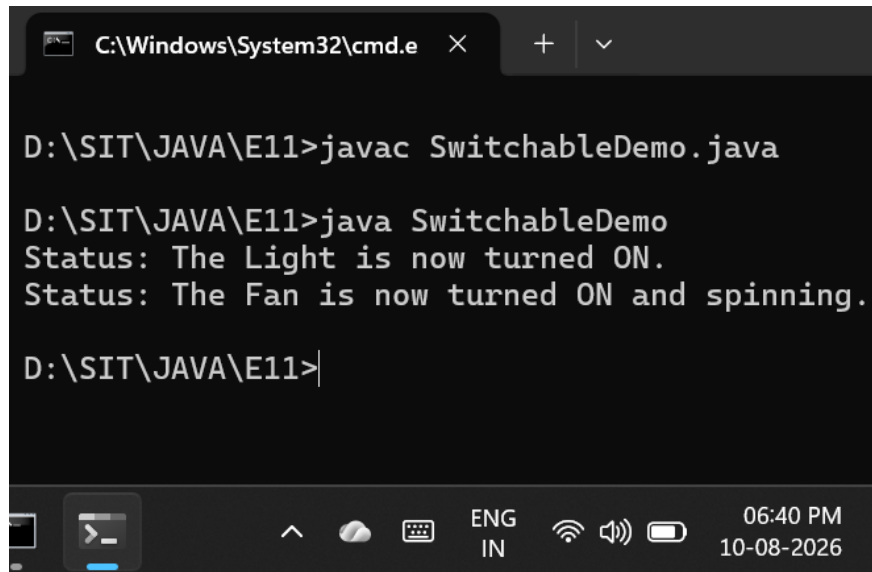
class Fan implements Switchable {
    public void turnOn() {
        System.out.println("Status: The Fan is now turned ON and spinning.");
    }
}

public class SwitchableDemo {
    public static void main(String[] args) {

        Switchable livingRoomLight = new Light();
        Switchable ceilingFan = new Fan();

        livingRoomLight.turnOn();
        ceilingFan.turnOn();
    }
}
```

Output



```
C:\Windows\System32\cmd.e X + v

D:\SIT\JAVA\E11>javac SwitchableDemo.java

D:\SIT\JAVA\E11>java SwitchableDemo
Status: The Light is now turned ON.
Status: The Fan is now turned ON and spinning.

D:\SIT\JAVA\E11>|
```

The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32\cmd.e'. The command history shows the compilation and execution of 'SwitchableDemo.java'. The output of the program is 'Status: The Light is now turned ON.' followed by 'Status: The Fan is now turned ON and spinning.' The taskbar at the bottom shows the system clock as 06:40 PM on 10-08-2026, along with various system icons like network, volume, and battery.